TECHNICAL WRITER ASSIGNMENT

AMOGH CG

ZOPDEV HSR layout, Bengaluru

Contents

TABLE OF FIGURES

1. API Reference: POST /deploy

1.1 Overview

The POST /deploy endpoint allows users to deploy infrastructure across multiple cloud providers (AWS, Azure, and GCP) using predefined YAML templates. This API enables automated multi-cloud deployment with a single request.

1.2 Endpoint

- **URL:** /deploy
- **Method:** POST
- **Headers:**

- Content-Type: application/json
- Authorization: Bearer <your_api_token>

1.3 Authentication

- This API requires authentication via a **Bearer Token**.
- Obtain an API token through the ZopDev authentication system.

1.4 Request Parameters

Body (JSON Format)

Parameter	Type	Required	Description
Template	string	Yes	YAML template defining cloud deployment.
Cloud providers	array	Yes	List of cloud providers (AWS, Azure, GCP).
Region	string	Yes	Deployment region (e.g., us-east-1).
Rollback Enabled	Boolean	No	Enables rollback in case of failure (default: false).
Tags	object	No	Key-value pairs for resource tagging.

Example Request (JSON)

```
{
  "template": "name: myApp\nresources:\n  - type: vm\n    provider: AWS\n properties:\n      instance_type: t2.micro\n      ami: ami-12345678",
```

```
  "cloud_providers": ["AWS", "GCP"],
  "region": "us-west-1",
  "rollback_enabled": true,
  "tags": { "environment": "staging", "team": "devops" }
}
```


1.5 Response Structure

Success Response (200 OK)

Field	Type	Description
deployment_id	string	Unique identifier for the deployment.
status	string	Deployment status (in_progress, success, failed).
logs_url	string	URL to view deployment logs.

Example Success Response


```
{
  "deployment_id": "abc12345",
  "status": "in_progress",
  "logs_url": "https://zopdev.com/logs/abc12345"
}
```


Error Response (400 Bad Request)

Field	Type	Description
error	string	Error message.

Example Error Response


```
{
  "error": "Invalid YAML format in template."
}
```


Error Response (401 Unauthorized)


```
{
  "error": "Invalid API token. Please authenticate."
```

```
}
```

Error Response (500 Internal Server Error)

```
{
  "error": "Deployment failed due to an internal server error."
}
```

2. Quick-Start Guide: Kubernetes Deployment

2.1 Create a Deployment YAML Template

To define a Kubernetes deployment, create a deployment.yaml file with the following structure:

```
tapiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-app-container
        image: my-app-image:latest
        ports:
```

```
      - containerPort: 80
```

Modify name, image, replicas, and containerPort as needed.

2.2 Deploy Using CLI

Use kubectl to apply the deployment:

kubectl apply -f deployment.yaml

Verify the deployment:

kubectl get deployments

2.3 Monitor Logs

To check logs for a running pod:

kubectl get pods

kubectl logs <pod-name>

To follow real-time logs:

kubectl logs -f <pod-name>

2.4 Manage Rollbacks

Check rollout history:

kubectl rollout history deployment my-app

Rollback to the previous version:

kubectl rollout undo deployment my-app

2.5 Web Dashboard Monitoring

To access the Kubernetes dashboard:

kubectl proxy

Then, navigate to:

http://localhost:8001/api/v1/namespaces/kube-system/services/https:kubernetes-dashboard:/proxy/

Use the dashboard to monitor deployments, view logs, and manage rollbacks.

3. Creative Flow Diagram- Feature Workflow

Steps:

1. **YAML Template Creation**
   - Define the configuration (e.g., resources, services).
   - Validate syntax and parameters.
2. **API Call**
   - Send YAML to deployment API.
   - API processes the request and triggers deployment.
3. **Deployment Process**
   - Infrastructure components are provisioned.
   - Services and dependencies are configured.
4. **Logging & Monitoring**
   - Capture logs (success, errors, warnings).
   - Send logs to a monitoring system (e.g., ELK, CloudWatch).
5. **Rollback (if needed)**
   - Detect failures via health checks.
   - Trigger rollback to the last stable state.

(NAPKIN AI generated Flow chart)

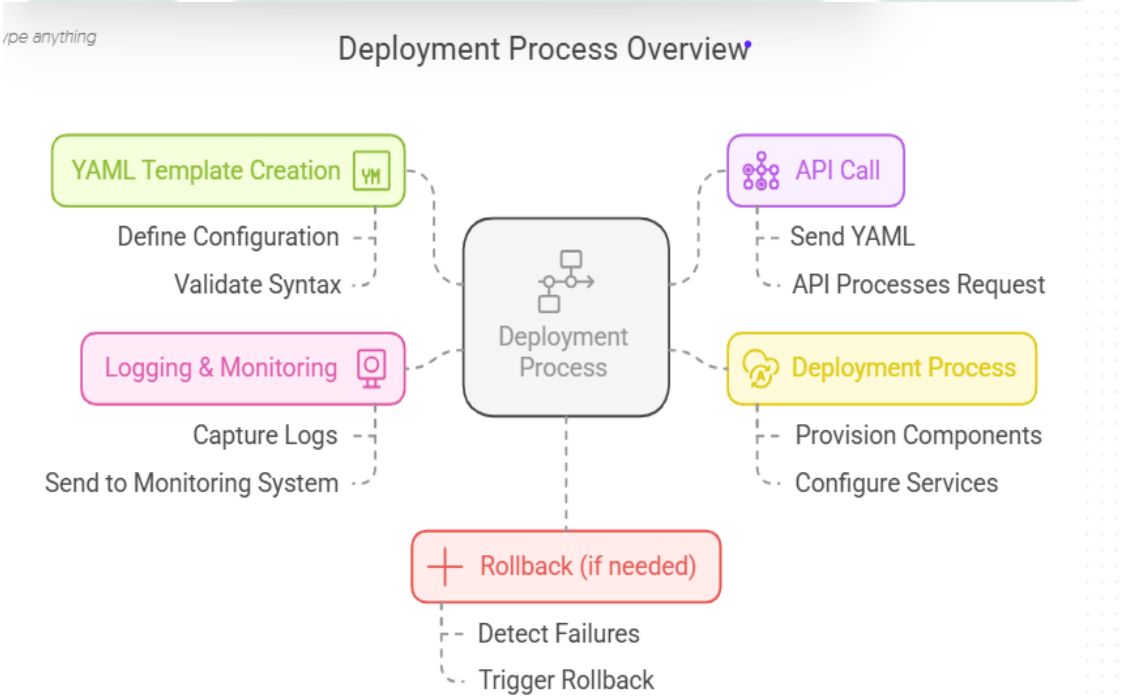

Figure 1

For LUICD chart link. Click here

https://lucid.app/lucidchart/8019cf11-1ad3-4b91-9a3f-237c05673600/edit?viewport_loc=-5112%2C-1920%2C6902%2C3006%2C0_0&invitationId=inv_f20260e4-ae00-402c-a3d4-d2cd843eb5a8

4. Documentation Strategy

To ensure clarity and usability, the documentation for this feature should be **structured, concise, and developer-friendly**. The following strategy outlines how to achieve this:

4.1 Structure

- **Overview:** Briefly explain the feature, its purpose, and key benefits.
- **Prerequisites:** List required dependencies, tools, and setup steps.
- **Step-by-Step Guide:** Provide a **Quick-Start Guide** with examples.
- **API Reference:** Detailed endpoint descriptions, request/response formats, and error codes.
- **Flow Diagram:** Visual representation of the process for quick understanding.
- **Troubleshooting & FAQs:** Common issues, solutions, and debugging tips.

4.2 Tools & Format

- **Markdown** (for GitHub, GitLab, or static site generators like MkDocs).
- **PDFs** (for easy offline access).
- **Lucidchart or Draw.io** (for flow diagrams).

4.3 Interactive Elements

- **Code Snippets** (copy-paste ready examples).
- **API Sandbox** (Postman collection or Swagger UI for testing).
- **Embedded Diagrams** (interactive or downloadable formats).

This approach ensures **clarity, accessibility, and ease of use** for developers.

5. Critique and Suggested Improvements

5.1 Evaluation of Structure, Clarity, and Completeness

The existing documentation provides a **concise and functional** explanation of ConfigMaps in Kubernetes. However, it **lacks context for beginners** and **misses deeper troubleshooting guidance** for experienced users. Below is a breakdown of key areas for improvement:

Strengths:

1. **Concise Definition:** The definition of ConfigMaps is clear and succinct.
2. **Straightforward Example Usage:** The documentation provides a direct example, which is useful for hands-on learners.
3. **Common Issues Section:** A brief troubleshooting section helps address basic errors.

5.2 Areas for Improvement:

5.2.1 Lack of Context for Beginners

▪ The documentation does not explain **why** ConfigMaps are useful, how they compare to other configuration methods (e.g., Secrets or environment variables in the Pod Spec), or when to use them.
▪ Suggested improvement: Add a **"Why Use ConfigMaps?"** subsection before the examples to provide better context.

5.2.2 Unstructured Example Usage

- The **example jumps into commands** without explaining their significance.
- YAML formatting **lacks indentation clarity**, making it harder for beginners to read.
- Suggested improvement: **Break down the example into smaller steps**, with comments explaining what each part does.

5.2.3 Limited Troubleshooting Guidance

The **Common Issues** section lacks details on debugging failed Pod startups due to missing ConfigMaps.

Suggested improvement: Provide **kubectl describe pod output** examples for debugging and add **solutions for resolving these errors**.

5.2.4 Missing Reference to YAML Validation

No mention of **how to validate YAML before applying it**, which can lead to syntax errors.

Suggested improvement: Add **a YAML linting tool recommendation** or a quick kubectl apply --dry-run=client command example.

6. Rewritten Documentation Section: "Common Issues"

Common Issues and Troubleshooting.

6.1. Missing Keys in the ConfigMap.

Problem: If a key in the ConfigMap is referenced but does not exist, the Pod will fail to start.

Example Error:

```sh

Error: container has incomplete environment variables ...

**Solution:**

- Check if the key exists in the ConfigMap using:
- kubectl get configmap game-config -o yaml
- If missing, update the ConfigMap:
- kubectl create configmap game-config --from-literal=player=anonymous

## 6.2 Improper Permissions

**Problem:** The user does not have the correct permissions to access ConfigMaps.
**Example Error:**

Error from server (Forbidden): configmaps "game-config" is forbidden

**Solution:**

- Check permissions with:
- kubectl auth can-i get configmaps --namespace=default
- If unauthorized, request access from an administrator or use:
- kubectl create rolebinding configmap-access --clusterrole=view --user=<your-user>

## 6.3 YAML Formatting Errors

**Problem:** Incorrect YAML formatting can prevent the Pod from starting.
**Solution:** Use a YAML validation tool before applying configurations:
```

- Validate locally:
- yamllint game-config.yaml
- Use Kubernetes' built-in validation:
- kubectl apply -f game-config.yaml --dry-run=client

Justification for Changes:

- More specific errors and solutions-help developers troubleshoot efficiently.
- Adding command outputs-improves debugging clarity.
- Introducing YAML validation- prevents common syntax mistakes.

6.4 Content Strategy Feedback

To integrate this documentation into a multi-cloud orchestration portal like Zop.dev, the following enhancements are recommended:

6.4.1 Contextual Cross-Linking to Zop.dev Workflows

▪ Instead of presenting Kubernetes documentation in isolation, link it to real-world use cases within Zop.dev.
▪ Example: If Zop.dev supports multi-cloud ConfigMaps synchronization, provide a step-by-step guide for integrating Kubernetes ConfigMaps across AWS, GCP, and Azure.

6.4.2 Interactive YAML Validation and ConfigMap Generator

▪ Include an interactive YAML playground ,where users can input key-value pairs and generate a valid ConfigMap YAML file.
▪ Justification: Reduces syntax errors,and simplifies onboarding.

6.4.3 Beginner vs. Advanced View Options

▪ Offer a "Simplified View" for beginners (step-by-step) and an "Advanced View" for power users (concise CLI commands).
▪ Justification: Enhances accessibility for different experience levels.

6.4.4 Dynamic Troubleshooting Assistant

▪ Implement a troubleshooting bot or flowchart that suggests solutions based on error messages.

▪ Justification: Reduces debugging time and improves support scalability.

6.4.5 Summary of Enhancements :

- Expanded beginner context: Added explanations on when/why to use ConfigMaps.
- Structured examples: Improved YAML readability with comments and indentation.
- Deeper troubleshooting: Added real error messages and debugging commands.
- Interactive feature suggestion: YAML validator to prevent formatting errors.
- Integration strategy: Connected Kubernetes ConfigMaps to multi-cloud workflows in Zop.dev.

Final Summary of Improvements:

Add context on why ConfigMaps are useful.
Break down the example to explain key components.
Include a verification command after ConfigMap creation.
Expand troubleshooting with real debugging steps

INDEX